## Learning Objectives

This lab lets you practice problem-solving using recursion.

## Estimated Size

1 program file: `recursion.py`, 3 functions.

## Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

## 0. Basics of Recursion

In Lecture 19, you've learned about recursion, where a function calls itself somewhere in its body. Recursion is a different way of thinking and is an elegant way to solve some computational problems. For recursion to work correctly, it must have a base case where it no longer calls itself, and each recursion must make progress towards the base case to avoid an infinite recursion. Although a lot of the simple examples of recursion (i.e. factorial, Fibonacci) can be easily solved without recursion, for other problems, recursion remains the simplest and most elegant solution. Examples include drawing fractals, the Hanoi tower game, computing permutations, and working with recursive data structures such as trees and graphs.

## 1. Implement function `max_recursive` (5 points)

Write a function `max_recursive(lst)` that returns the **maximum element** of a list of integers using **recursion only**.

- Must be solved using **recursion only**.
- No loops of any kind.
- The list may contain integers or floats only. All the values will be positive.

### Example

```
max_recursive([3, 10, 2, 8, 6]) # returns 10
max_recursive([10, 2, 8, 6])    # returns 10
max_recursive([2, 8, 6])        # returns 8
max_recursive([8, 6])           # returns 8
max_recursive([6])              # returns 6 -> base case
```

```
max_recursive([])               # returns 0 -> base case
```

## 2. Implement function `sum_lists_recursive` (5 points)

Write a function `sum_lists_recursive(lst1, lst2)` that takes **two lists of integers of equal length** and returns the **summation** of the elements of the two input lists. You **must** solve this using **recursion only** — no loops allowed.

- Assume both lists have the same length.
- Must be solved using **recursion**.
- No `for` loops, no list comprehensions.
- You may use slicing (e.g., `lst[1:]`).

### Example

```
sum_lists_recursive([1, 2, 3], [4, 5, 6]) # returns 21
sum_lists_recursive([2, 3], [5, 6])       # returns 16
sum_lists_recursive([3], [6])             # returns 9
sum_lists_recursive([],[])                # returns 0 -> base case
```

## 3. Implement function `funky` (5 points)

Some mathematical functions are defined recursively, so it's natural to implement them using recursion as well. For example, below is the mathematical definition of a funky function which takes an integer `n` (can be either positive or negative) as a parameter:

$$f(n) = \begin{cases} 1 & \text{if } n \text{ is } 0 \text{ or } 1 \\ 2 * f(n//2) & \text{if } n \text{ is an even number} \\ 1 + 2 * f(n+1) & \text{otherwise} \end{cases}$$

Write a recursive function called `funky` that implements the above function. Below are some example test cases for you to verify your implementation:

```
funky(2)=2, funky(10)=74, funky(50)=554, funky(-10)=50, funky(-50)=418
```

## 4. Implement function `permutations` - Optional (0 points)

Given a list of items, say `['AA','BB','CC']`, a permutation is an ordering of the items. Here, we want to compute all possible permutations of the items. Mathematically, if the input has `n` items (`n>=1`), the number of all possible permutations is `n!` (factorial). For the example above, there are 6 possible permutations: `['AA','BB','CC']`, `['AA','CC','BB']`, `['BB','AA','CC']`, `['BB','CC','AA']`,

`['CC','AA','BB'], ['CC','BB','AA']`

This is a classic problem that demonstrates the **recursive** way of thinking. Let's think about how to solve this problem by hand. To begin, if the list has just `1` item, then there is just `1` possible permutation -- the item itself. This is the base case of the recursion. If there are `n` items, we would loop over each item, pull it to the front (call it the `front_item`) so it becomes the first item of a permutation. Then we are left with `(n-1)` remaining items to permutate. If we can somehow get all permutations of these remaining `(n-1)` items, then inserting the `front_item` to each permutation would give us permutations of the original `n` items where `front_item` is the first item.

How do we get all permutations of `(n-1)` items? **That's the recursive part:** this is just like the original problem, but with one less item. As a concrete example, look at the 3-item list above. We would loop over each of them, starting with `'AA'` -- call it the `front_item`. Then we are left with just 2 items: `['BB','CC']`, and by recursion, we found out there are 2 possible permutations of them: `['BB','CC'],['CC','BB']`. So when this returns, we insert `'AA'` at the front of each, resulting in `['AA','BB','CC'], ['AA','CC','BB']`. Going back, we now have `'BB'` as the `front_item`, and are left with `['AA','CC']` as the remaining items to permutate. So we do the same, resulting in `['BB','AA','CC'], ['BB','CC','AA']`. Finally, we have `'CC'` as the `front_item`, resulting in permutations `['CC','AA','BB'], ['CC','BB','AA']`.

Write a function called `permutations`. It takes **a list** `lis` as the parameter, and it returns **a list of lists**, where each item in the returned list represents a different permutation of the input items. You can assume the input list has at least 1 item and that all items are unique. Below is a guideline to help you implement this function, and the recommended names for variables:

1. First, write the base case: if `lis` has only 1 item, return `[lis]` as the return value, because this is the only possible permutation.
2. Create an empty list, say `retlis`, representing the list to be returned in the end.
3. Loop over the index range of `lis` by using `for i in range(len(lis)):`
   In the loop body, assign the current item you are iterating to a variable called `front_item` by using `front_item = lis[i]`. Then make a new list, call it `remaining`, which contains the remaining items -- this can be done by using slicing or list comprehension (i.e. how to make a new list from `lis` that contains all its items except the one at index `i`?)
4. Now comes the recursive part: call `permutations` (i.e. the function itself) with `remaining` as the argument. Remember, this function call should return all permutations of the items in `remaining`. Next, loop over the returned permutations, insert `front_item` at the beginning of each permutation, and append the result to `retlis`.
5. Return `retlis` at the end of the function.

Because recursion is a new concept to many of you, please take your time to understand the logic above and ask a staff member to help you if needed. Let's first reason about whether the recursion can terminate successfully: it has a base case (when the input list has only 1 item); and each recursive call deals with a smaller list (1 less item) so it progresses towards the base case. Therefore the recursion should terminate successfully.

Next, test your base case, say, `print(permutations(['AA']))`. This should return `[['AA']]`. Note the double square brackets, as the return value must be a list of lists.

Now test a 2-item input: `print(permutations(['AA','BB']))`. It should return `[['AA','BB'], ['BB','AA']]`. If you encounter any error of incorrect results, use the debugger to help you run the code line by line. You can set a **breakpoint** at the first line of your function, then run the program in debug mode. Once it pauses at the breakpoint, use **Step Over** to run each line and check variable values. When you come to the recursive call, you can click **Step Into** which allows you to go into the function call (in this case, the function itself!) and continue to examine the execution result.

Once you are done with a 2-item input, test your function with a 3-item input and so on. For an `n`-item input, the length of the list returned by your function should be `n!` (factorial). So it grows rapidly as `n` becomes large.

## 5. (Common Paragraph) Submission to Gradescope

Log in to Gradescope, select **Lab 11: Recursion (Code)**, and submit the required file.

Also remember to submit the lab questionnaire. To do so, open the **Lab 11 (Questionnaire)** Google Doc link (available on Piazza), make a copy so you can edit the file in Google Doc. When you are done, download it as a PDF and submit it to Gradescope. Select all your group members when submitting.

## (Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.